

Университет ИТМО

Расширенный курс по backend-разработке в среде Node.js

Домашняя работа №4

Технический дизайн микросервисной архитектуры

Платформа для фитнес-тренировок и здоровья

Выполнила:

Яковенко Ксения

Группа: БР1.1

2026 г.

1. Цель работы

Цель работы - спроектировать переход монолитного REST API фитнес-платформы к микросервисной архитектуре. В текущем монолите уже выделены функциональные области: авторизация и профиль пользователя, каталог тренировок, планы тренировок, прогресс пользователя, метрики тела и блог. В рамках работы эти области разделяются на отдельные сервисы с собственными базами данных.

Главная идея разделения: каждый сервис владеет только своей частью данных, а другие сервисы обращаются к ним через внутренний REST API. Прямые внешние ключи между базами разных сервисов не используются: вместо этого хранятся внешние идентификаторы, например `userId`, `workoutId` или `planId`.

2. Исходный монолит и границы разбиения

В текущей версии есть следующие сущности и таблицы: `users`, `user_profiles`, `revoked_tokens`, `workouts`, `workout_types`, `workout_type_map`, `difficulty_levels`, `training_plans`, `plan_workouts`, `user_training_plans`, `workout_sessions`, `body_metrics`, `blog_posts`, `blog_categories` и `post_categories`..

Такое разбиение хорошо ложится на предметные области, поэтому микросервисы выделяются не по техническим слоям, а по бизнес-возможностям.

Область	Что было в монолите	Что станет сервисом
Пользователи и доступ	Регистрация, логин, профиль, роли, JWT	Identity Service
Каталог тренировок	Тренировки, типы тренировок, уровни сложности	Workout Catalog Service
Планы тренировок	Планы, состав плана, запись пользователя на план	Training Plan Service
Прогресс	Фактические тренировки и метрики тела	Progress Service
Контент	Блог о здоровье и питании, категории	Content Service
Файлы	Загрузка картинок/видео для тренировок и постов	Media Service
Уведомления	Будущие уведомления о записи, прогрессе, публикациях	Notification Service

3. Предлагаемая микросервисная архитектура

Внешний клиент не обращается к микросервисам напрямую. Все публичные запросы идут через API Gateway / BFF. Gateway проверяет JWT, извлекает userId и role, после чего передает запрос в нужный сервис. Это упрощает клиентскую часть и не раскрывает внутреннюю структуру системы наружу.

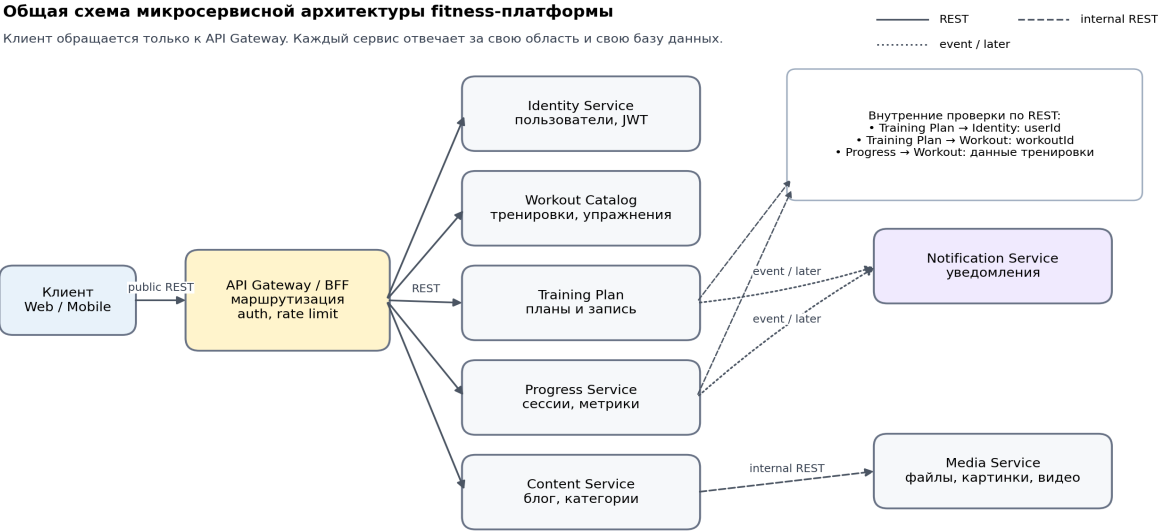


Рисунок 1 - Общая схема микросервисной архитектуры

Сервис	Ответственность	Примеры публичных endpoint-ов
API Gateway / BFF	Единая точка входа, маршрутизация, проверка токена, rate limit, CORS	/auth/*, /workouts/*, /training-plans/*, /users/me/*
Identity Service	Регистрация, вход, пользователи, профили, роли, блокировки, revoked tokens	/auth/register, /auth/login, /users/me, /users/me/profile
Workout Catalog Service	Каталог тренировок, фильтрация, детализация, типы и уровни сложности	/workouts, /workouts/{id}, /metadata/workout-types
Training Plan Service	Планы тренировок, состав планов, запись пользователя, статус прохождения	/training-plans, /training-plans/{id}/enroll, /users/me/training-plans
Progress Service	История тренировок пользователя, workout sessions, body metrics	/users/me/workout-sessions, /users/me/body-metrics
Content Service	Блог, категории, публикация и редактирование материалов	/blog/posts, /blog/categories
Media Service	Хранение метаданных файлов и выдача ссылок на изображения/видео	/media/uploads
Notification Service	Создание уведомлений пользователям. На первом этапе может работать только как внутренний сервис	internal only

4. Разделение баз данных

Для соблюдения database-per-service каждый доменный сервис получит отдельную базу данных. Внутри базы можно использовать обычные связи и внешние ключи. Между базами разных сервисов внешние ключи не создаются, потому что сервисы должны быть слабо связаны.

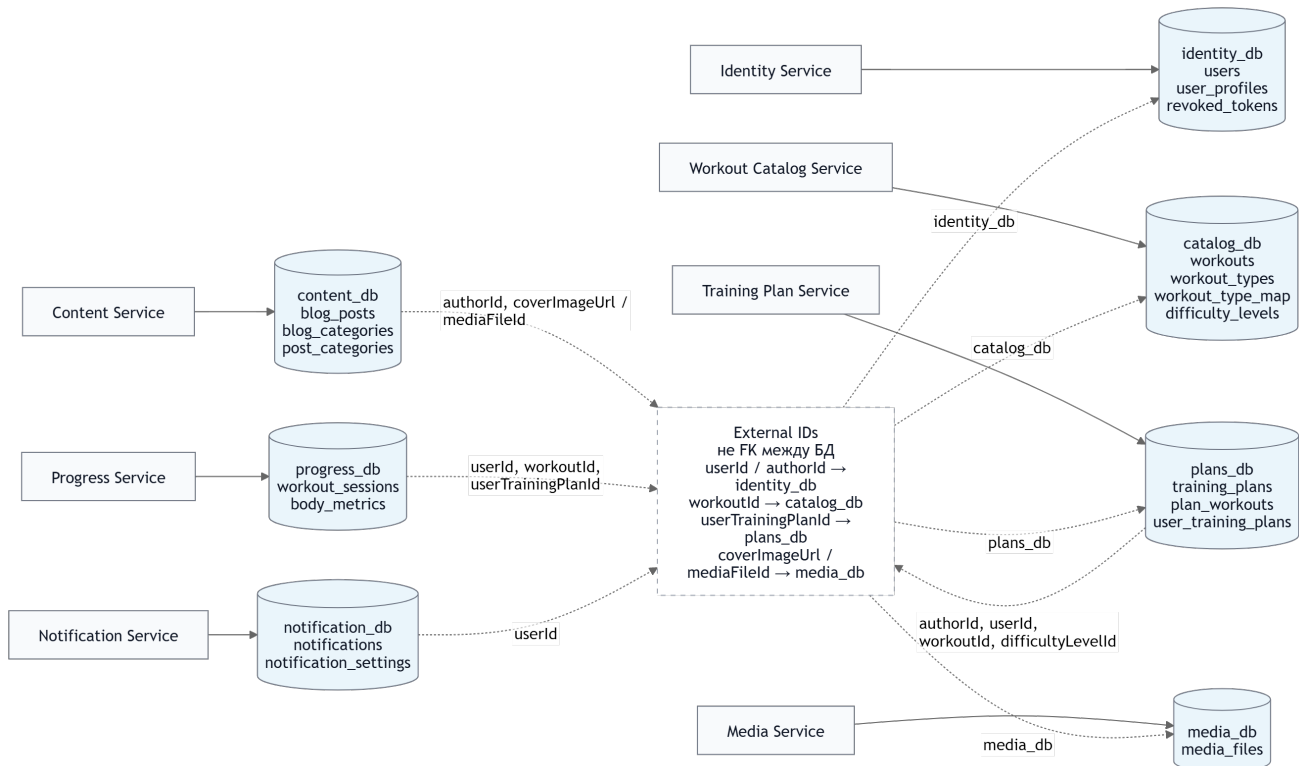


Рисунок 2 - Разделение данных по сервисам

База данных	Таблицы	Владелец данных	Внешние id
identity_db	users, user_profiles, revoked_tokens	Identity Service	—
catalog_db	workouts, workout_types, workout_type_map, difficulty_levels	Workout Catalog Service	authorId как id пользователя, без FK в identity_db
plans_db	training_plans, plan_workouts, user_training_plans	Training Plan Service	authorId, userId, workoutId, difficultyLevelId
progress_db	workout_sessions, body_metrics	Progress Service	userId, workoutId, userTrainingPlanId
content_db	blog_posts, blog_categories, post_categories	Content Service	authorId, coverImageUrl / mediaFileId
media_db	media_files	Media Service	ownerId
notification_db	notifications, notification_settings	Notification Service	userId

В ERD-схемах поля userId, authorId, workoutId, difficultyLevelId, userTrainingPlanId, planId, coverImageUrl и mediaFileId в базах других сервисов считаются внешними идентификаторами или внешними ссылками, но не физическими foreign key между разными базами данных..

WORKOUT_SESSIONS		
uuid	id	PK
uuid	user_id	
uuid	workout_id	
uuid	user_training_plan_id	
datetime	started_at	
datetime	finished_at	
int	duration_min	
int	calories_burned	
string	status	
text	comment	
datetime	created_at	
datetime	updated_at	

BODY_METRICS		
uuid	id	PK
uuid	user_id	
float	weight	
float	chest	
float	waist	
float	hips	
datetime	measured_at	
datetime	created_at	
datetime	updated_at	

Схема показывает фактический прогресс пользователя: выполненные тренировки и метрики тела. В исходном проекте нет отдельной таблицы `exercise_results`, поэтому в `progress_db` остаются `workout_sessions` и `body_metrics`. Поля `user_id`, `workout_id` и `user_training_plan_id` являются внешними идентификаторами, так как пользователь, тренировка и план находятся в других сервисах.

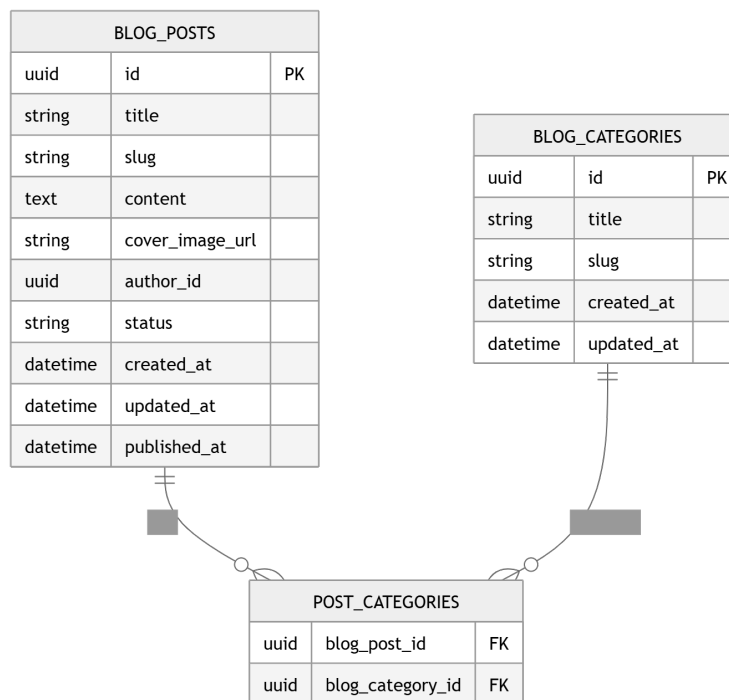


Схема относится к блогу и категориям публикаций. Названия `blog_posts`, `blog_categories` и `post_categories` сохранены из исходного проекта. В исходном монолите обложка поста хранится как `cover_image_url`. При выделении Media Service это поле можно заменить на `media_file_id` или использовать как ссылку на файл, обслуживаемый Media Service.

MEDIA_FILES		
uuid	id	PK
string	file_name	
string	mime_type	
string	storage_url	
int	size_bytes	
uuid	uploaded_by_user_id	
datetime	created_at	

Схема описывает новую базу Media Service. В исходном монолите отдельной таблицы media_files не было: ссылки на изображения и видео хранились прямо в сущностях, например video_url у workouts и cover_image_url у blog_posts. В микросервисной версии файлы можно вынести в отдельный сервис, чтобы тренировки и блог хранили только ссылку или media_file_id.

NOTIFICATIONS		
uuid	id	PK
uuid	user_id	
string	type	
string	title	
string	message	
string	status	
datetime	created_at	
datetime	sent_at	

NOTIFICATION_SETTINGS		
uuid	id	PK
uuid	user_id	
boolean	email_enabled	
boolean	push_enabled	
boolean	workout_reminders_enabled	
datetime	updated_at	

Схема описывает новую базу Notification Service. В исходном монолите уведомления не были отдельной сущностью, но в микросервисной архитектуре их удобно вынести отдельно. Notification Service хранит уведомления и настройки пользователя, а user_id используется как внешний идентификатор пользователя из Identity Service.

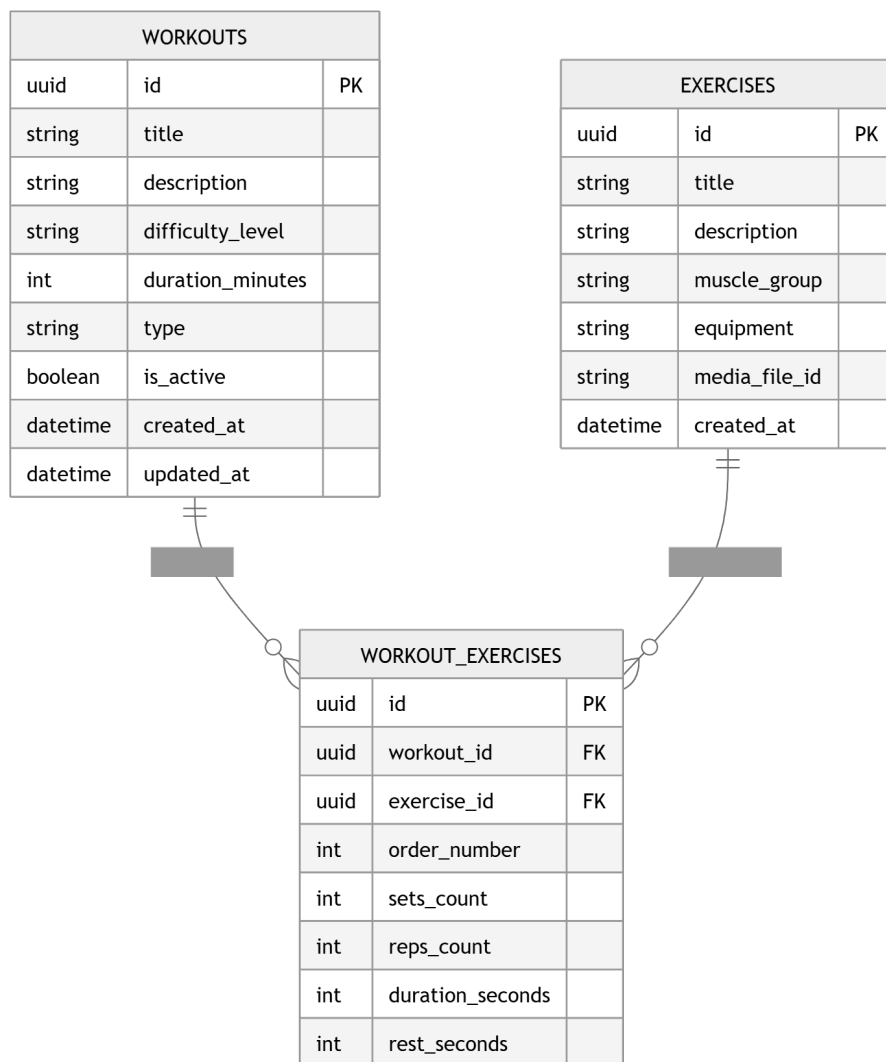


Схема описывает каталог тренировок. В исходном проекте нет отдельной таблицы `exercises`, поэтому состав тренировки хранится в самой таблице `workouts` через описание, инструкции и видео. Типы тренировок вынесены в `workout_types`, а связь многие-ко-многим оформляется через `workout_type_map`. Поле `authorId` считается внешним идентификатором пользователя из Identity Service.

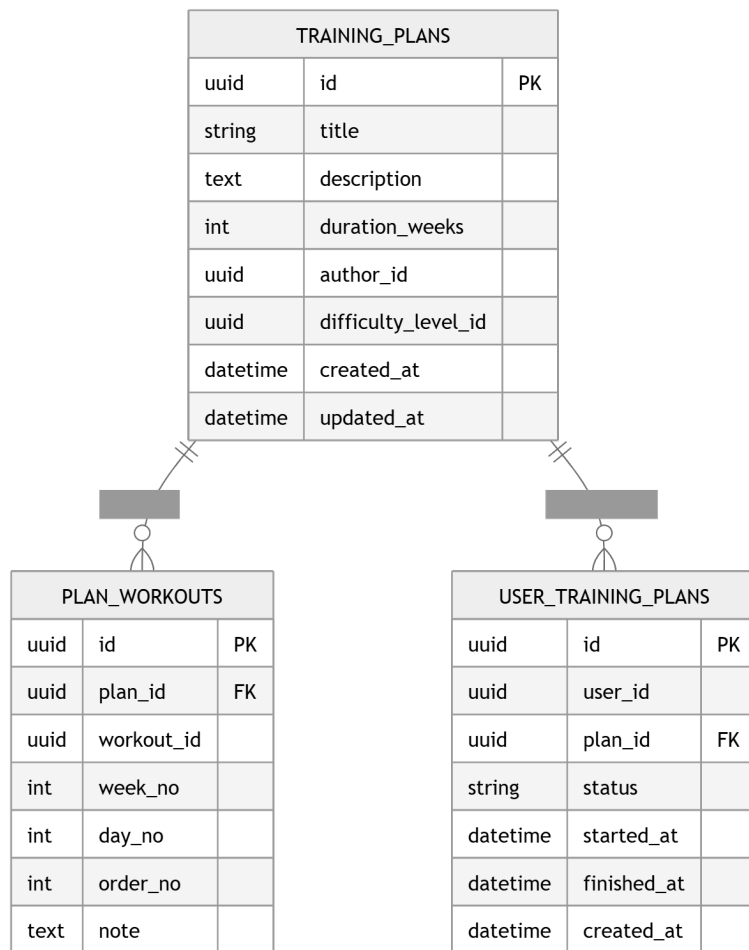


Схема описывает тренировочные планы, состав плана и запись пользователя на план. Название `user_training_plans` сохранено из исходного проекта. Внутри `plans_db` можно связать `training_plans` с `plan_workouts` и `user_training_plans`. Поля `user_id`, `workout_id`, `author_id` и `difficulty_level_id` считаются внешними идентификаторами из других сервисов, поэтому они не являются физическими FK между разными базами данных.

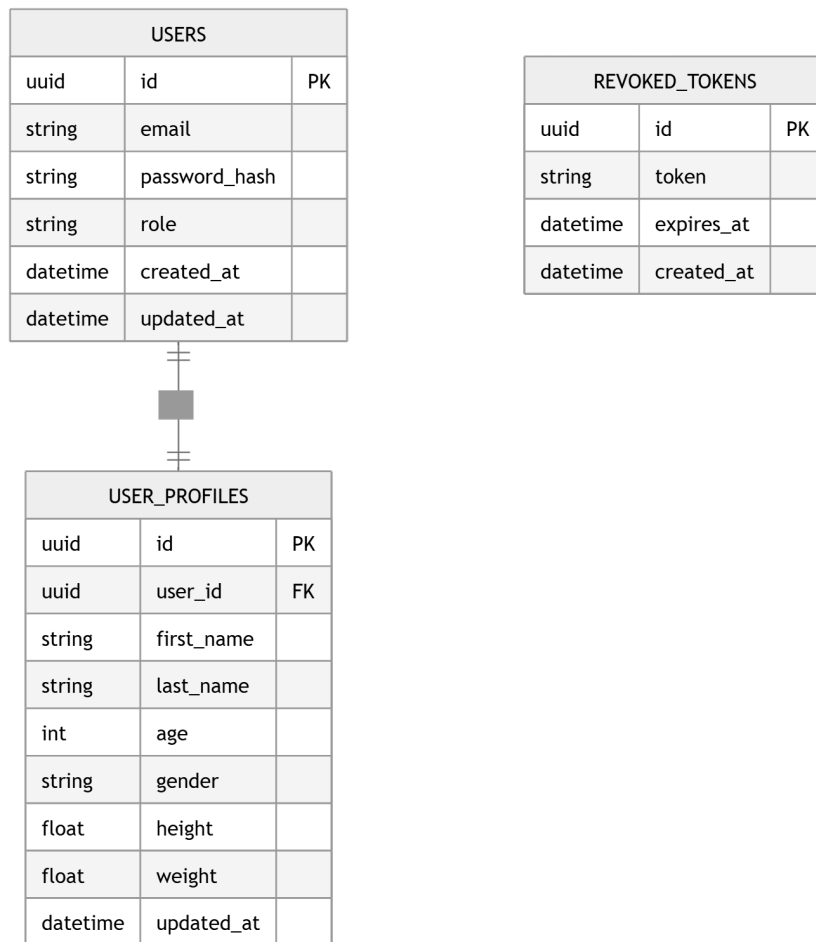


Схема показывает данные Identity Service: пользователей, профили и отозванные JWT-токены. В ответ возвращаются accessToken, refreshToken и краткая информация о пользователе. При выходе или отзыве токена он может попадать в таблицу revoked_tokens. Таблица user_profiles связана с пользователем через user_id внутри identity_db. Таблица revoked_tokens не связывается с users напрямую, потому что хранит сам отозванный токен и срок его действия.

Возможная проблема такого подхода - данные становятся распределенными. Например, в Progress Service хранится workoutId, но само название тренировки лежит в Catalog Service. Для решения используется либо синхронный запрос в Catalog Service, либо хранение небольшого snapshot-а названия тренировки в момент создания workout session. Я решила, что для данной работы подойдет и синхронный REST-запрос.

5. Межсервисное взаимодействие

Основной способ взаимодействия на этапе ДЗ4 и ЛР2 - синхронный REST внутри docker/network. Для внутренних запросов используется отдельный набор endpoint-ов с префиксом /internal. Доступ к ним закрывается с помощью X-Service-Token, чтобы внешний клиент не мог вызывать межсервисные методы напрямую.

Сценарий	Сервисы	Тип связи	Комментарий
Проверка JWT	Gateway -> Identity	REST	Gateway получает userId и role из токена
Запись на план	Training Plan -> Catalog	REST	Проверяются тренировки, входящие в план

Создание workout session	Progress -> Catalog	REST	Проверяется, что workoutId существует
Обновление прогресса плана	Progress -> Training Plan	REST	После завершения тренировки обновляется процент прохождения
Уведомление о записи	Training Plan -> Notification	Event / REST	Notification Service сохраняет уведомление в notification_db. Сейчас связь можно оставить через REST, а в Д35 заменить на RabbitMQ/Kafka.
Публикация поста с картинкой	Content -> Media	REST	В исходном монолите у blog_posts используется coverImageUrl. В микросервисной версии Content Service может хранить mediaFileId или ссылку, полученную от Media Service.

6. Основные пользовательские сценарии

6.1 Регистрация и вход

1. Клиент отправляет запрос POST /auth/register или POST /auth/login в API Gateway. 2. Gateway перенаправляет запрос в Identity Service. 3. Identity Service создает пользователя или проверяет пароль. 4. В ответ возвращаются accessToken, refreshToken и краткая информация о пользователе. Остальные сервисы не хранят email и пароль пользователя.

6.2 Поиск и просмотр тренировки

Клиент отправляет GET /workouts с фильтрами по типу, уровню сложности и длительности. Gateway перенаправляет запрос в Workout Catalog Service. Сервис работает только со своей catalog_db и возвращает список тренировок с пагинацией.

6.3 Запись пользователя на тренировочный план

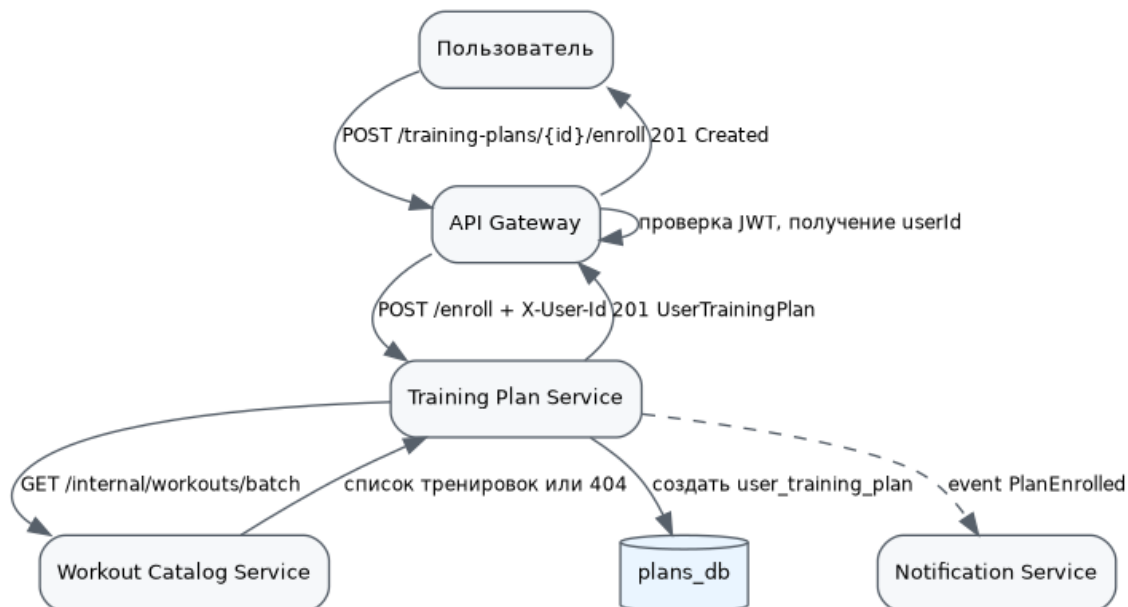


Рисунок 3 - Последовательность записи на тренировочный план

При записи пользователя Training Plan Service создает запись в таблице user_training_plans в plans_db. Если нужно проверить состав плана или тренировку, сервис обращается к Workout Catalog Service. Если пользователь уже записан на этот план, возвращается ошибка 409 Conflict.

6.4 Фиксация выполненной тренировки

Клиент отправляет POST /users/me/workout-sessions. Gateway добавляет userId из токена. Progress Service проверяет workoutId через Catalog Service, сохраняет workout_session в progress_db и при наличии userTrainingPlanId сообщает Training Plan Service, что прогресс по плану нужно обновить.

6.5 Добавление метрик тела

Метрики тела полностью относятся к Progress Service. Запросы GET/POST/PATCH/DELETE /users/me/body-metrics не требуют обращения к другим сервисам, кроме проверки пользователя на уровне Gateway.

6.6 Работа с блогем

Публичные пользователи могут читать опубликованные посты и категории. Создавать, редактировать и удалять посты могут trainer/admin. Проверка роли выполняется через Gateway и/или Identity Service, а сами blog_posts, blog_categories и post_categories хранятся только в content_db.

7. Внутренний OpenAPI для межсервисного взаимодействия

Для Д34 отдельно спроектированы внутренние endpoint-ы. Полная спецификация вынесена в файл openapi-microservices.yaml. Ниже приведен фрагмент с основными группами методов.

Endpoint	Метод	Назначение	Основные ошибки
/identity/internal/tokens/verify	POST	Проверить JWT и вернуть userId, role	401, 422, 500
/identity/internal/users/{userId}	GET	Получить краткие данные пользователя	401, 403, 404, 500
/catalog/internal/workouts/{workoutId}	GET	Проверить существование тренировки и получить summary	401, 404, 500
/catalog/internal/workouts/batch	POST	Получить несколько тренировок по ids	401, 422, 500
/plans/internal/user-training-plans/{userTrainingPlanId}	GET	Получить данные пользовательского плана	401, 404, 500
/plans/internal/user-training-plans/{userTrainingPlanId}/progress	PATCH	Обновить прогресс плана после workout session	401, 404, 409, 422, 500
/progress/internal/users/{userId}/summary	GET	Получить сводку прогресса пользователя	401, 404, 500
/notifications/internal/notifications	POST	Создать уведомление	401, 422, 503, 500
/media/internal/files	POST	Создать запись о файле и вернуть mediaFileId	401, 422, 500
/media/internal/files/{fileId}	GET	Получить метаданные или проверить существование файла	401, 404, 500

```

securitySchemes:
  serviceToken:
    type: apiKey
    in: header
    name: X-Service-Token

/catalog/internal/workouts/{workoutId}:
  get:
    summary: Get workout short data by id
    responses:
      '200': Workout exists
      '401': Invalid or missing service token
      '404': Workout not found
      '500': Internal server error

```

8. Обработка ошибок

Код	Когда возвращается	Пример
400 Bad Request	Неверный формат запроса	Некорректный JSON
401 Unauthorized	Нет пользовательского JWT или внутреннего X-Service-Token	Запрос без токена
403 Forbidden	Роль не позволяет выполнить действие	Обычный user пытается создать тренировку
404 Not Found	Ресурс не найден в сервисе-владельце	workoutId не существует
409 Conflict	Конфликт состояния	Повторная запись на уже активный план
422 Validation Error	Данные не прошли валидацию	durationMin меньше 1
503 Service Unavailable	Зависимый сервис временно недоступен	Notification Service не отвечает
500 Internal Server Error	Неожиданная ошибка сервера	Ошибка БД

9. План перехода от монолита к микросервисам

Шаг	Действие	Результат
1	Зафиксировать текущий OpenAPI монолита и основные сценарии Postman	Понятно, что должно сохраниться после разделения
2	Выделить Identity Service и перенести users/profiles/tokens	Остальные сервисы перестают работать с паролями и токенами
3	Выделить Workout Catalog Service	Каталог тренировок становится независимым
4	Выделить Training Plan Service	Планы и записи пользователей переносятся в plans_db
5	Выделить Progress Service	Сессии тренировок и метрики тела переносятся в progress_db
6	Выделить Content, Media и Notification Service	Блог, файлы и уведомления отделяются от core-функций
7	Настроить API Gateway и внутренние URL сервисов	Клиент продолжает видеть единый API
8	Добавить docker-compose для всех сервисов и отдельных БД	Можно запускать всю систему одной командой
9	В ДЗ5/ЛР3 заменить часть REST-вызовов событиями RabbitMQ/Kafka	Сервисы становятся менее связанными

10. Нефункциональные требования

Требование	Решение в архитектуре
Безопасность	Публичный доступ только через Gateway, JWT для пользователей, X-Service-Token для внутренних вызовов, роли user/trainer/admin. Пароли хранятся только в Identity Service.
Масштабируемость	Сервисы можно масштабировать независимо. Например, Catalog и Content чаще читаются, Progress чаще пишет данные.
Отказоустойчивость	Ошибки зависимых сервисов возвращаются как 503. Для некритичных операций, например уведомлений, можно использовать очередь и повторные попытки.
Согласованность данных	Между сервисами используется eventual consistency. В учебной версии часть операций синхронная, но без распределенных транзакций.
Наблюдаемость	Gateway добавляет correlationId, сервисы пишут структурированные логи. Метрики можно собирать через Prometheus.
Развертывание	Каждый сервис имеет собственный Dockerfile. Общий docker-compose поднимает сервисы и их базы данных.

В результате монолитное приложение фитнес-платформы разделяется на несколько микросервисов с понятными границами ответственности. Основные данные распределяются по отдельным базам, а связи между сервисами реализуются через внутренние REST API и внешние идентификаторы.